An Interior Point Method for Separable Problems

Mostafa Abdalla Ramzi Zakhama

December 11, 2022

Abstract

We formulate an interior point algorithm for separable convex min-max problems with positive semi-definite side constraints.

1 Problem

Consider the following min-max optimisation problem,

$$\min_{\boldsymbol{x}} \max\{f_i(\boldsymbol{x}) \,|i \in \mathcal{O}\} \text{ subject to: } \{f_i(\boldsymbol{x}) \leq 0 \,|i \in \bar{\mathcal{O}}\} \tag{1}$$

where $\mathcal{O}$ is the objective set and $\bar{\mathcal{O}}$ is its complement with respect to the set of all responses $\{1, 2, \ldots, n\}$.

In structural optimisation the design vector $\boldsymbol{x}$ may be partitioned as,

$$\boldsymbol{x} = \big(\boldsymbol{x}_1 | \boldsymbol{x}_2 | \ldots \boldsymbol{x}_b\big)^t, \tag{2}$$

where b is the number of *blocks* and each subvector $\boldsymbol{x}_j$ has length m_j. The total number of design variables is,

$$m = \sum_{j=1}^{b} m_j. \tag{3}$$

Corresponding to the partition of the design vector is postulated the following structure of the response,

$$f_i(\boldsymbol{x}) = \sum_{j=1}^{b} f_{ij}(\boldsymbol{x}_j). \tag{4}$$

The functional form in (4) is known as a separable function. Such a form arises often as a consequence of the use of structural approximations. Classically, approximation are constructed such that all block sizes $m_j = 1$. This corresponds to each block devolving into a scalar. In this work we consider the more general case $m_j \geq 1$. Needless to say the virtue of separability is

evident only when all the blocksizes are small compared to the total number of variables.

The constraints posed in (1) are *response* constraints and generally depend on all the subvectors in $\boldsymbol{x}$. In addition to these some *side* constraints are applied. We assume that the side constraints are in semi-definite form,

$$\boldsymbol{X}_j(\boldsymbol{x}_j) \succeq \boldsymbol{0}. \tag{5}$$

The order r_j of each semi-definite constraint is determined by the size of the corresponding square matrix $\boldsymbol{X}_j$. The simplest form of side constraints are bound constraints of the form,

$$x_j \leq x_j^U, \quad x_j \geq x_j^L. \tag{6}$$

The min-max form in (1) is not smooth. To avoid this we use the bound formulation. We introduce a scalar upper bound on the objective, z, and a boolean vector $\boldsymbol{o}$ such that,

$$o_i = 1 \;\; \forall i \in \mathcal{O}, \text{ and } o_i = 0 \;\; \forall i \in \bar{\mathcal{O}}. \tag{7}$$

The optimisation problem, with side constraints, may be formulated as,

$$\min_{z,\boldsymbol{x}} z, \tag{8}$$

subject to:

$$\boldsymbol{f}(\boldsymbol{x}) - z\boldsymbol{o} \leq \boldsymbol{0} \tag{9}$$

$$\boldsymbol{X}_j(\boldsymbol{x}_j) \succeq \boldsymbol{0}, \quad j = 1, 2, \ldots, b \tag{10}$$

The problem is further formulated using slack variables as,

$$\min_{z,\boldsymbol{x}} z, \tag{11}$$

subject to:

$$\boldsymbol{f}(\boldsymbol{x}) - z\boldsymbol{o} + \boldsymbol{s} = \boldsymbol{0} \tag{12}$$

$$-\boldsymbol{X}_j(\boldsymbol{x}_j) + \boldsymbol{S}_j = 0, \quad j = 1, 2, \ldots, b \tag{13}$$

$$\boldsymbol{s} \geq \boldsymbol{0} \tag{14}$$

$$\boldsymbol{S}_j \succeq \boldsymbol{0}. \tag{15}$$

The Lagrangian is formulated as,

$$\mathcal{L} = z + \boldsymbol{y}^t\big(\boldsymbol{f} - z\boldsymbol{o} + \boldsymbol{s}\big) + \sum_{j=1}^{b} \boldsymbol{Y}_j \cdot \big(-\boldsymbol{X}_j(\boldsymbol{x}_j) + \boldsymbol{S}_j\big) - \mu\left(\sum_{i=1}^{n} \log s_i + \sum_{j=1}^{b} \log\det \boldsymbol{S}_j\right) \tag{16}$$

The underlined terms are logarithmic barrier function and $\mu > 0$ is a penalty parameter. The barrier functions are used to replace the constraints on the slack variables. Setting the derivative with respect to all variables to zero, we obtain, after some simplification,

$$
\begin{aligned}
z :& 1 - \boldsymbol{o}^t \boldsymbol{y} = 0 && (17)\\
x :& \boldsymbol{G}\boldsymbol{y} - \boldsymbol{c} = \boldsymbol{0} && (18)\\
s :& \boldsymbol{s} * \boldsymbol{y} = \mu \boldsymbol{1} && (19)\\
S_j :& \boldsymbol{S}_j \boldsymbol{Y}_j = \mu \boldsymbol{I} && (20)\\
y :& \boldsymbol{f} - z\boldsymbol{o} + \boldsymbol{s} = \boldsymbol{0} && (21)\\
Y_j :& - \boldsymbol{X}_j(\boldsymbol{x}_j) + \boldsymbol{S}_j = \boldsymbol{0}. && (22)
\end{aligned}
$$

The vector c contains the contribution of the side constraints in the form,

$$\boldsymbol{c} = \left(\boldsymbol{c}_1 | \boldsymbol{c}_2 | \ldots \boldsymbol{c}_b\right)^t, \tag{23}$$

where,

$$\boldsymbol{c}_j = \boldsymbol{\mathcal{G}}_j^t \cdot \boldsymbol{Y}_j, \tag{24}$$

and $\boldsymbol{\mathcal{G}}_j$ is the third order tensor $\partial_{\boldsymbol{x}_j} \boldsymbol{X}_j$. The matrix $\boldsymbol{G}$ has dimensions $m \times n$ and each column contains the gradient of one the corresponding response. The total order of the semi-definite program is,

$$r = \sum_{j=1}^{b} r_j + n. \tag{25}$$

Linearising these optimality conditions we obtain,

$$
\begin{aligned}
z :& - \boldsymbol{o}^t d\boldsymbol{y} = r_z && (26a)\\
x :& \boldsymbol{H} d\boldsymbol{x} + \boldsymbol{G} d\boldsymbol{y} - \sum_{j=1}^{b} \boldsymbol{\mathcal{G}}_j^t \cdot d\boldsymbol{Y}_j = \boldsymbol{r_x} && (26b)\\
s :& d\boldsymbol{s} * \boldsymbol{y} + \boldsymbol{s} * d\boldsymbol{y} = \boldsymbol{r_s} && (26c)\\
S_j :& d\boldsymbol{S}_j \boldsymbol{Y}_j + \boldsymbol{S}_j d\boldsymbol{Y}_j = \boldsymbol{R}_{\boldsymbol{S}_j} && (26d)\\
y :& \boldsymbol{G}^t d\boldsymbol{x} - dz\boldsymbol{o} + d\boldsymbol{s} = \boldsymbol{r_y} && (26e)\\
Y_j :& - \boldsymbol{\mathcal{G}}_j \cdot d\boldsymbol{x}_j + d\boldsymbol{S}_j = \boldsymbol{R}_{\boldsymbol{Y}_j}. && (26f)
\end{aligned}
$$

The matrix $\boldsymbol{H}$ is a combination of second order derivatives of the responses and the side constraints. Since the responses are separable, $\boldsymbol{H}$ is block-diagonal.

$$\boldsymbol{H}_j = \partial_{\boldsymbol{x}_j} \partial_{\boldsymbol{x}_j} \left(\sum_{i=1}^{n} y_i f_{ij} - \boldsymbol{Y}_j \cdot \boldsymbol{X}_j \right) \tag{27}$$

The system of equations (26a) defining the search direction will be solved by successive elimination of variables to obtain a Schur complement form.

We start by eliminating the slack and dual variables, $d\boldsymbol{S}_j$ and $d\boldsymbol{Y}_j$, associated with the side constraints.

$$d\boldsymbol{S}_j = \boldsymbol{\mathcal{G}}_j \cdot d\boldsymbol{x}_j + \boldsymbol{R}_{\boldsymbol{Y}_j}, \tag{28}$$

and,

$$d\boldsymbol{Y}_j = \langle \boldsymbol{R}_{\boldsymbol{S}_j} - \boldsymbol{S}_j^{-1} d\boldsymbol{S}_j \boldsymbol{Y}_j \rangle \tag{29}$$

where $\langle \bullet \rangle$ is the symmetric part of a matrix. Next we eliminate the slack vector $\boldsymbol{s}$,

$$d\boldsymbol{s} = (\boldsymbol{r}_{\boldsymbol{s}} - \boldsymbol{s} * d\boldsymbol{y})/\boldsymbol{y} \tag{30}$$

Now the system is reduced to the form,

$$x : \boldsymbol{B} d\boldsymbol{x} + \boldsymbol{G} d\boldsymbol{y} = \boldsymbol{r}'_{\boldsymbol{x}} \tag{31a}$$

$$y : \boldsymbol{G}^t d\boldsymbol{x} - \boldsymbol{C} d\boldsymbol{y} - dz\boldsymbol{o} = \boldsymbol{r}'_{\boldsymbol{y}} \tag{31b}$$

$$z : -\boldsymbol{o}^t d\boldsymbol{y} = r_z. \tag{31c}$$

Or in matrix form,

$$\begin{pmatrix} \boldsymbol{B} & \boldsymbol{G} & \boldsymbol{0} \\ \boldsymbol{G}^t & -\boldsymbol{C} & -\boldsymbol{o} \\ \boldsymbol{0} & -\boldsymbol{o} & \boldsymbol{0} \end{pmatrix} \begin{pmatrix} d\boldsymbol{x} \\ d\boldsymbol{y} \\ dz \end{pmatrix} = \begin{pmatrix} \boldsymbol{r}'_{\boldsymbol{x}} \\ \boldsymbol{r}'_{\boldsymbol{y}} \\ r_z \end{pmatrix} \tag{32}$$

where,

$$\boldsymbol{B}_j = \boldsymbol{H}_j + \boldsymbol{\mathcal{G}}_j \cdot \boldsymbol{X}_j^{-1} \cdot \boldsymbol{Y}_j \cdot \boldsymbol{\mathcal{G}}_j^t, \tag{33}$$

$$\boldsymbol{C} = \boldsymbol{s}/\boldsymbol{y}, \tag{34}$$

$$\boldsymbol{r}'_{\boldsymbol{x}} = \boldsymbol{r}_{\boldsymbol{x}} + \sum_{j=1}^{b} \boldsymbol{\mathcal{G}}_j^t \cdot \boldsymbol{X}_j^{-1} \cdot \boldsymbol{R}_{\boldsymbol{S}_j}, \tag{35}$$

$$\boldsymbol{r}'_{\boldsymbol{y}} = \boldsymbol{r}_{\boldsymbol{y}} - \boldsymbol{r}_{\boldsymbol{s}}/\boldsymbol{y}. \tag{36}$$

The system (31a) may be solved directly or by first eliminating either $d\boldsymbol{x}$ or $d\boldsymbol{y}$. For a separable system the matrix $\boldsymbol{B}$ is block-diagonal and it is attractive to eliminate $d\boldsymbol{x}$ to obtain the further reduced form,

$$\begin{pmatrix} \boldsymbol{D} & \boldsymbol{o} \\ \boldsymbol{o} & \boldsymbol{0} \end{pmatrix} \begin{pmatrix} d\boldsymbol{y} \\ dz \end{pmatrix} = \begin{pmatrix} -\boldsymbol{r}''_{\boldsymbol{y}} \\ -r_z \end{pmatrix} \tag{37}$$

where,

$$\boldsymbol{D} = \boldsymbol{C} + \boldsymbol{G}^t \boldsymbol{B}^{-1} \boldsymbol{G}, \tag{38}$$

$$\boldsymbol{r}''_{\boldsymbol{y}} = \boldsymbol{r}'_{\boldsymbol{y}} - \boldsymbol{G}^t \boldsymbol{B}^{-1} \boldsymbol{r}'_{\boldsymbol{x}}. \tag{39}$$

The matrix $\boldsymbol{D}$ is positive definite and one may be tempted to further eliminate $d\boldsymbol{y}$. Unfortunately, for small values of the penalty parameter μ the matrix is ill-conditioned. This holds true even if all constraints are active.

To deal with this ill-conditioning, the vector $d\boldsymbol{y}$ is permuted to start from the least active constraint to the most active. In fact, we make sure the last element in the permuted vector corresponds to the critical response; that is the response in the objective set that have the highest value. The permuted vector now is partitioned as,

$$\boldsymbol{P}\boldsymbol{y} = \left(d\boldsymbol{y}_I | dy_c\right)^t \tag{40}$$

The permuted residual also,

$$\boldsymbol{P}\boldsymbol{r}''_{\boldsymbol{y}} = \left(\boldsymbol{r}''_I | r''_c\right)^t \tag{41}$$

and the permuted matrix

$$\boldsymbol{P}\boldsymbol{D}\boldsymbol{P}^t = \begin{pmatrix} \boldsymbol{D}_{I,I} & \boldsymbol{d}_c \\ \boldsymbol{d}_c^t & d_{cc} \end{pmatrix} \tag{42}$$

Thus the reduced system (37) is written as,

$$\begin{pmatrix} \boldsymbol{D}_{I,I} & \boldsymbol{d}_c & \boldsymbol{o}_I \\ \boldsymbol{d}_c^t & d_{cc} & 1 \\ \boldsymbol{o}_I^t & 1 & \boldsymbol{0} \end{pmatrix} \begin{pmatrix} d\boldsymbol{y}_I \\ dy_c \\ dz \end{pmatrix} = \begin{pmatrix} -\boldsymbol{r}''_I \\ -r''_c \\ -r_z \end{pmatrix} \tag{43}$$

Now we can proceed to eliminate $d\boldsymbol{y}_I$, and end up with the following 2×2 system that can be solved exactly,

$$\begin{pmatrix} d & q \\ q & -e \end{pmatrix} \begin{pmatrix} dy_c \\ dz \end{pmatrix} = \begin{pmatrix} -r'''_c \\ -r'_z \end{pmatrix} \tag{44}$$

where,

$$d = d_{cc} - \boldsymbol{d}_c^t \boldsymbol{D}_{I,I}^{-1} \boldsymbol{d}_c, \tag{45}$$

$$q = 1 - \boldsymbol{d}_c^t \boldsymbol{D}_{I,I}^{-1} \boldsymbol{o}_I, \tag{46}$$

$$e = \boldsymbol{o}_I^t \boldsymbol{D}_{I,I}^{-1} \boldsymbol{o}_I, \tag{47}$$

$$r'''_c = r''_c - \boldsymbol{d}_c^t \boldsymbol{D}_{I,I}^{-1} \boldsymbol{r}''_I, \tag{48}$$

$$r'_z = r_z - \boldsymbol{o}_I^t \boldsymbol{D}_{I,I}^{-1} \boldsymbol{r}''_I, \tag{49}$$

Algorithm 1 internal procedures

```
function DualityGap( )
    gap ← (s + ds) ∗ (y + dy)
    for j = 1 . . . m do
        gap ← gap + Side_j.DualityGap( )
    end for
    return gap
end function
```

```
procedure CalculateResiduals(μ, f, G)
    ds ← μ1 − s ∗ y − ds ∗ dy
    dy ← f − oz + ds
    for j = 1 . . . m do
        dx_j ← 0
        Side_j.CalculateResiduals(μ, x_j, dx_j)
    end for
    dx ← dx − G dy
    dx′ ← B^{−1} dx
    dy ← dy + G^t dx′
    (dy_I | dy_c)^t ← P dy
    dy′_I ← D^{−1}_{I,I} dy_I
    dz ← −1 + o^t y
    dz ← dz + o_I^t dy′_I
    dy_c ← dy_c + d_c^t dy′_I
end procedure
```

```
procedure UpdateIncrements(G)
    f ← ed + q^2
    dz ← (ddz + q dy_c) / f
    dy_c ← (dy_c − q dz) / d
    dy_I ← dy_I − d_c dy_c − o_I dz
    dy_I ← D_{I,I}^{-1} dy_I
    dy ← P^t (dy_I | dy_c)^t
    ds ← (ds − s ∗ dy) / y
    dx ← dx − G dy
    dx ← B^{-1} dx
    for j = 1 ... m do
        Side_j.UpdateIncrements(dx_j)
    end for
end procedure
```

```
procedure CalculateHessian(G, B)
    for j = 1 ... m do
        B_j ← 0
        Side_j.CalculateHessian(B_j)
        for i = 1 ... n do
            B_j ← B_j + y_i ∂^2 f_ij / ∂x_j^2
        end for
    end for
    for i = 1 ... n do
        G_i ← ∇_x f_i
    end for
    G' ← B^{-1} G
    D ← (s / y)
    D ← D + G^t G'
    ( D_{I,I}   d_c )
    ( d_c^t     d   ) ← P D P^t
    (o_I | 1)^t ← P o
    d'_c ← D_{I,I}^{-1} d_c
    o'_I ← D_{I,I}^{-1} o_I
    d ← d − d_c^t d'_c
    q ← 1 − d_c^t o'_I
    e ← o_I^t o'_I
end procedure
```

```
function StepSize(α_p, α_d)
    α_p ← StepSize(z, dz)
    α_p ← StepSize(s, ds)
    α_d ← StepSize(y, dy)
    α_p ← StepSize(x, dx)
    for j = 1 ... m do
        Side_j.StepSize(α_{p_j}, α_{d_j})
        α_p ← min{α_p, α_{p_j}}
        α_d ← min{α_d, α_{d_j}}
    end for
end function
```

```
procedure UpdateVariables(α_p, α_d)
    x ← x + α_p dx
    s ← s + α_p ds
    y ← y + α_d dy
    z ← z + α_p dz
    for j = 1 ... m do
        Side_j.UpdateVariables(α_p, α_d)
    end for
end procedure
```

```
procedure CreatPermutationMatrix(f, o, P)
    for i = 1 ... n do
        index'[i] ← i
    end for
    ShellSort(f, index')
    i_c ← 1
    while o[index'[i_c]] ≠ 1 do
        i_c ← i_c + 1
    end while
    for i = 1 ... (i_c − 1) do
        index[i] ← index'[i]
    end for
    for i = (i_c + 1) ... n do
        index[i] ← index'[i]
    end for
    index[n] ← index'[i_c]
    P ← 0
    for i = 1 ... n do
        P[i, index[i]] ← 1
    end for
end procedure
```

```
procedure SOLVER(x)
    INITIALISE(1, x)
    Δ ← DUALITYGAP( )
    while Δ > tol do
        OBJECTIVEFUNCTION(x, f, G, B)
        CREATPERMUTATIONMATRIX(f, o, P)
        μ ← Δ/(n + m)
        CALCULATEHESSIAN(G, B)
        CALCULATERESIDUALS(β_p μ, f, G)
        UPDATEINCREMENTS(G)
        Δ̃ ← DUALITYGAP( )
        β̃_c ← CORRECTORMULTIPLIER(Δ, Δ̃)
        CALCULATERESIDUALS(β̃_c μ, f, G)
        UPDATEINCREMENTS(G)
        STEPSIZE(α_p, α_d)
        UPDATEVARIABLES(α_p, α_d)
        Δ ← DUALITYGAP( )
    end while
end procedure
```